

1. Mục tiêu

- Hiểu rõ vai trò của Web Server và Reverse Proxy trong kiến trúc một hệ thống backend thực tế.
- Nắm vững các khái niệm về IP, Domain, Port (80, 443), SSL/TLS, và file `/etc/hosts`.
- Biết cấu hình NGINX làm **Web Server** để phục vụ file tĩnh (ví dụ: bản build của frontend).
- Biết cấu hình NGINX làm **Reverse Proxy** đứng trước một cặp Frontend + Backend (định tuyến theo path, ví dụ `/` cho FE và `/api/` cho BE).
- Biết dùng `upstream` để NGINX điều hướng và cân bằng tải đến **nhiều backend/nhiều node server** khác nhau.
- Nắm được các cấu hình hay gặp trong thực tế: trở tới container FE/BE khi chạy Docker, cấu trúc một `server` block chuẩn, giới hạn kích thước body request tối đa, cấu hình domain và mô phỏng SSL, hiểu vì sao một chứng chỉ SSL được trình duyệt tin cậy.
- Chuẩn bị tốt cho phần thuyết trình và demo.

2. Nội dung cần nghiên cứu và chuẩn bị

Nội dung được xây dựng theo đúng lộ trình từ đơn giản đến phức tạp: NGINX phục vụ file tĩnh → Reverse Proxy cho một cặp FE/BE → Upstream cho nhiều backend/nhiều node → các cấu hình thực tế hay gặp (container, body size, domain, SSL).

2.1. Khái niệm cơ bản

- **Web Server:** Phần mềm phục vụ các file tĩnh (HTML, CSS, JS, hình ảnh) trực tiếp cho trình duyệt (ví dụ: NGINX, Apache).
- **Reverse Proxy:** Một server đặt phía trước một hoặc nhiều server khác (web server, API backend), nhận request từ client rồi chuyển tiếp đến server phía sau phù hợp. Giúp tăng cường bảo mật (ẩn backend thật), cân bằng tải, quản lý SSL tập trung.
- **IP (Internet Protocol):** Địa chỉ duy nhất của một thiết bị trên mạng.
- **Domain (Tên miền):** Tên dễ nhớ để truy cập website thay vì địa chỉ IP (ví dụ: google.com).
- **Port:** Cổng giao tiếp mà các ứng dụng dùng để nhận/gửi dữ liệu (ví dụ: 80 cho HTTP, 443 cho HTTPS).
- **SSL/TLS:** Giao thức mã hóa dữ liệu truyền tải giữa client và server, đảm bảo an toàn thông tin.
- **File `/etc/hosts`:** File trên hệ điều hành dùng để ánh xạ tên miền với địa chỉ IP cục bộ, hữu ích để giả lập domain khi phát triển/kiểm thử mà không cần mua domain hay cấu hình DNS thật.

2.2. NGINX như một Web Server (phục vụ file tĩnh)

Đây là vai trò gốc và đơn giản nhất của NGINX, cần nắm trước khi học đến reverse proxy.

- **Cài đặt NGINX:** Hướng dẫn cài đặt trên Linux (Ubuntu/WSL).
- **File cấu hình:** `/etc/nginx/nginx.conf` và các file trong `/etc/nginx/sites-available` (được symlink sang `/etc/nginx/sites-enabled`).
- **Phục vụ file tĩnh:**
 - `root`: Thư mục gốc chứa file tĩnh trên server (ví dụ: `root /var/www/myapp;`).
 - `index`: File mặc định khi truy cập thư mục (ví dụ: `index index.html;`).
 - Ví dụ thực tế: phục vụ bản build của một ứng dụng frontend (React/Vue) sau khi `npm run build`.
 - `try_files $uri $uri/ /index.html;`: Cấu hình cần thiết cho các ứng dụng SPA (Single Page Application) để mọi route không tồn tại file vật lý đều fallback về `index.html`, cho phép client-

side routing hoạt động đúng.

2.3. NGINX làm Reverse Proxy cho một cặp Frontend – Backend

- **Kịch bản:** Một domain duy nhất phục vụ cả frontend lẫn backend API — frontend ở `/`, API ở `/api/`, tránh vấn đề CORS vì client luôn gọi cùng một origin.
- **location và proxy_pass:**
 - `location /` phục vụ file tĩnh của frontend (dùng `root/try_files` như mục 2.2).
 - `location /api/ { proxy_pass http://localhost:5000/; }` chuyển tiếp các request `/api/...` đến backend.
- **Các header quan trọng khi proxy:** Mặc định NGINX không tự chuyển tiếp thông tin gốc của request, cần khai báo:
 - `proxy_set_header Host $host;`
 - `proxy_set_header X-Real-IP $remote_addr;`
 - `proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;`
 - `proxy_set_header X-Forwarded-Proto $scheme;`
 - Giải thích: nếu không có các header này, backend sẽ thấy mọi request đều đến từ chính NGINX (127.0.0.1) thay vì IP thật của client, gây khó khăn khi cần log, rate-limit hoặc phân tích theo IP người dùng.

2.4. Upstream và nhiều Backend/Node Server

- **Vấn đề:** Khi một backend không đủ sức chịu tải, hoặc cần đảm bảo hệ thống không "chết" nếu một node gặp sự cố, ta cần nhiều bản backend (nhiều instance/nhiều node server khác nhau) cùng chạy song song.
- **upstream:** Định nghĩa một nhóm các backend server (có thể là nhiều container trên cùng máy, hoặc nhiều máy chủ/node khác nhau với địa chỉ IP khác nhau):

```
upstream my_backends {  
    server 10.0.0.11:5000;  
    server 10.0.0.12:5000;  
    server localhost:5001;  
}
```

- **Sử dụng trong location:** `proxy_pass http://my_backends;` thay vì trỏ trực tiếp đến một backend cụ thể.
- **Cân bằng tải (Load Balancing):** Mặc định NGINX dùng round-robin (lần lượt) giữa các server trong `upstream`. Có thể tìm hiểu thêm các chiến lược khác: `least_conn` (ưu tiên server ít kết nối nhất), `weight` (ưu tiên theo trọng số nếu các node có cấu hình phần cứng khác nhau).
- **Liên hệ buổi Docker Compose:** Đây chính là NGINX đứng trước nhiều instance backend đã tạo ở buổi trước (Buổi 7) — biến việc "chạy nhiều container backend" thành một hệ thống thực sự phân tải được truy cập qua một địa chỉ duy nhất.

2.5. Các cấu hình hay gặp trong thực tế

- **Cấu hình trỏ vào container Frontend:** Khi FE được đóng gói trong một container NGINX riêng (build xong copy vào image NGINX) hoặc chạy dưới dạng service Node, `proxy_pass` sẽ trỏ đến tên service

trong Docker network (ví dụ `proxy_pass http://frontend:80;`) thay vì `localhost`.

- **Cấu hình trỏ vào container Backend:** Tương tự, `proxy_pass http://backend:5000;` hoặc trỏ vào `upstream` gồm các tên service backend trong cùng Docker network (service discovery bằng tên, như đã học ở Buổi 7).
- **Một server block chuẩn thường gồm những gì:**
 - `listen`: Port lắng nghe (80 cho HTTP, 443 cho HTTPS).
 - `server_name`: Domain mà block này xử lý.
 - `location /`: Phục vụ frontend (file tĩnh hoặc proxy tới container FE).
 - `location /api/`: Proxy tới backend/upstream, kèm các `proxy_set_header` cần thiết.
 - `client_max_body_size`: Giới hạn kích thước body tối đa.
 - Cấu hình log riêng cho từng server (`access_log`, `error_log`) để dễ debug theo từng domain/ứng dụng.
- **Giới hạn kích thước body tối đa (`client_max_body_size`):** Mặc định NGINX chỉ cho phép body request tối đa 1MB, request lớn hơn (ví dụ upload file, ảnh) sẽ bị từ chối với lỗi `413 Request Entity Too Large`. Cần tăng giá trị này ở `http`, `server` hoặc `location` block tùy phạm vi áp dụng (ví dụ: `client_max_body_size 20M;`).

2.6. Cấu hình Domain và SSL (mô phỏng)

- **Giả lập domain bằng file hosts:** Thêm một dòng ánh xạ domain giả về địa chỉ local (ví dụ: `127.0.0.1 myapp.local`), sau đó cấu hình `server_name myapp.local;` trong NGINX để test như đang truy cập một domain thật, dù chưa có DNS/domain thật. **Lưu ý: file cần sửa và cách chạy NGINX sẽ khác nhau tùy môi trường, cần phân biệt rõ 3 trường hợp:**
 - **Trường hợp 1 — NGINX chạy trực tiếp trên Windows (bản NGINX for Windows), trình duyệt cũng ở Windows:** Sửa file `C:\Windows\System32\drivers\etc\hosts` bằng Notepad chạy với quyền Administrator (file này không cho sửa/lưu nếu mở quyền thường). Đây là trường hợp không dùng WSL/Linux chút nào — cả server lẫn client đều thuần Windows.
 - **Trường hợp 2 — NGINX chạy trong WSL/Ubuntu, nhưng truy cập bằng trình duyệt Windows (cách phổ biến nhất khi thực hành trên máy Windows):** Trình duyệt chạy trên Windows phân giải domain bằng file hosts của Windows, không phải file `/etc/hosts` bên trong WSL. Vì vậy dù NGINX chạy trong Linux, vẫn phải sửa `C:\Windows\System32\drivers\etc\hosts` như trường hợp 1, chứ sửa `/etc/hosts` trong WSL sẽ không có tác dụng gì với trình duyệt.
 - **Trường hợp 3 — Cả server lẫn client (ví dụ dùng curl) đều nằm trong cùng một môi trường Linux thuần (WSL hoặc Ubuntu server thật):** Sửa `/etc/hosts` bên trong Linux đó bằng `sudo nano /etc/hosts`.
 - **Lưu ý riêng về WSL2:** WSL2 mặc định tự động sinh lại file `/etc/hosts` bên trong mỗi lần khởi động (dựa theo cấu hình mạng), nên các dòng thêm thủ công có thể bị mất sau khi restart WSL. Nếu cần thêm dòng cố định vào `/etc/hosts` của WSL (trường hợp 3), có thể tắt tính năng tự sinh này bằng cách thêm `generateHosts = false` vào mục `[network]` trong file `/etc/wsl.conf`, rồi restart WSL.
- **Tạo chứng chỉ SSL tự ký (self-signed) để giả lập HTTPS local:**
 - Dùng lệnh `openssl req -x509 -nodes -newkey rsa:2048 -keyout myapp.key -out myapp.crt -days 365` để tạo cặp key/cert.
 - Cấu hình trong NGINX: `listen 443 ssl;`, `ssl_certificate myapp.crt;`, `ssl_certificate_key myapp.key;`.

- **Lưu ý quan trọng:** Chứng chỉ tự ký sẽ khiến trình duyệt cảnh báo "Not Secure"/"NET::ERR_CERT_AUTHORITY_INVALID" vì đây chỉ để test cục bộ, không thể dùng thật trên internet.
- **Trên server thật:** Dùng **Certbot** (Let's Encrypt) để lấy chứng chỉ SSL miễn phí và được trình duyệt tin cậy thật sự, ví dụ `certbot --nginx -d yourdomain.com`, kèm cơ chế tự động gia hạn (renew) trước khi hết hạn.
- **Vì sao một chứng chỉ SSL được trình duyệt tin cậy (chain of trust):**
 - Hệ điều hành/trình duyệt có sẵn danh sách **Root CA** (Certificate Authority) đáng tin cậy.
 - Root CA ký (hoặc ủy quyền cho Intermediate CA ký) chứng chỉ cho domain của bạn khi bạn chứng minh được quyền sở hữu domain đó (đây là việc Certbot/Let's Encrypt tự động hóa).
 - Khi trình duyệt kết nối, nó xác minh chữ ký của chứng chỉ ngược lên tới một Root CA có sẵn trong danh sách tin cậy (trust store) của hệ điều hành/trình duyệt — nếu xác minh được toàn bộ chuỗi (chain of trust) thì kết nối được coi là an toàn.
 - Chứng chỉ tự ký (self-signed) không có mắt xích nào trong chuỗi trên (tự làm "CA" cho chính mình) nên trình duyệt không có cách nào xác minh, do đó luôn bị cảnh báo.

3. Chuẩn bị thuyết trình và demo

- **Slide:** Giải thích các khái niệm Web Server, Reverse Proxy, IP, Domain, Port, SSL, `/etc/hosts`, và trình bày cấu trúc file cấu hình NGINX theo đúng lộ trình ở mục 2.
- **Demo — thực hiện tuần tự theo lộ trình, ưu tiên gắn với ứng dụng thật của nhóm (frontend + backend + nhiều instance backend từ Buổi 6, 7):**
 1. **NGINX là Web Server:** Build một bản frontend tĩnh (hoặc trang HTML đơn giản), cấu hình NGINX phục vụ file này qua `root/index`.
 2. **NGINX là Reverse Proxy FE + BE:** Thêm một backend API (của nhóm), cấu hình `location /api/` proxy tới backend, kiểm tra frontend gọi API qua cùng domain không bị lỗi CORS. Minh họa các `proxy_set_header` bằng cách log thử `X-Real-IP` ở phía backend.
 3. **Upstream nhiều backend/nhiều node:** Chạy 2 (hoặc nhiều hơn) instance backend (có thể tái sử dụng từ demo Buổi 7), cấu hình `upstream`, gọi lặp lại API và chứng minh request được phân phối luân phiên giữa các instance (ví dụ mỗi backend trả về một ID/port khác nhau để phân biệt).
 4. **Cấu hình container FE/BE (nếu nhóm đã Docker hóa ở buổi trước):** Sửa `proxy_pass` để trỏ tới tên service trong Docker network thay vì `localhost`, demo NGINX cũng chạy trong cùng Docker network hoặc trỏ ra ngoài qua port map.
 5. **client_max_body_size:** Thử upload một file/gửi body lớn hơn 1MB khi chưa cấu hình để thấy lỗi 413, sau đó thêm `client_max_body_size` và thử lại thành công.
 6. **Domain + SSL mô phỏng:** Sửa file hosts đúng theo môi trường đang dùng (làm rõ với người xem đây là Windows native, WSL+trình duyệt Windows, hay Linux thuần — theo 3 trường hợp ở mục 2.6) để trỏ một domain giả về local, tạo chứng chỉ tự ký bằng `openssl`, cấu hình `listen 443 ssl;`, truy cập thử bằng HTTPS và chỉ ra cảnh báo "not trusted" trên trình duyệt. Giải thích ngắn gọn vì sao (chain of trust) và cách khắc phục thật sự trên server production bằng Certbot/Let's Encrypt.
- **Thời gian:** Phân bổ thời gian hợp lý cho lý thuyết và demo, ưu tiên thời lượng cho bước 2 và 3 (trọng tâm reverse proxy + upstream).

4. Phân công công việc và ước lượng thời gian (Gợi ý)

Task	Người thực hiện (Gợi ý)	Ước lượng (Point)
Nghiên cứu khái niệm cơ bản: Web server, Reverse Proxy, IP, Domain, Port, SSL, <code>/etc/hosts</code>	TV1	3
Nghiên cứu NGINX web server (phục vụ file tĩnh, SPA fallback)	TV2	2
Nghiên cứu Reverse Proxy cho FE/BE và các <code>proxy_set_header</code>	TV2	3
Nghiên cứu <code>upstream</code> , load balancing nhiều backend/node	TV1	3
Nghiên cứu cấu hình thực tế: container FE/BE, <code>client_max_body_size</code> , cấu trúc server block chuẩn	TV3	3
Nghiên cứu domain giả lập, SSL tự ký bằng <code>openssl</code> , Certbot và chain of trust	TV3	3
Chuẩn bị nội dung slide thuyết trình	Nhóm trưởng	2
Cấu hình NGINX và chuẩn bị toàn bộ kịch bản demo (6 bước ở mục 3)	Cả nhóm	5
Tổng hợp, kiểm tra và chạy thử demo	Cả nhóm	2
Luyện tập thuyết trình và demo	Cả nhóm	1
Tổng cộng		27

5. Tài liệu tham khảo

- **Tài liệu chính thức:** NGINX Documentation (nginx.org/en/docs/), tài liệu `openssl req`, tài liệu Certbot (certbot.eff.org).
- **Bài viết:** Các bài hướng dẫn cấu hình NGINX làm web server, reverse proxy, load balancing với `upstream`; bài viết giải thích chain of trust của SSL/TLS.
- **Video:** Các video hướng dẫn cài đặt, cấu hình NGINX và Let's Encrypt/Certbot trên server thật.

6. Ghi chú quan trọng

- **Hiểu rõ luồng request:** Khi dùng NGINX, luôn hình dung rõ luồng request từ client → NGINX → (FE tĩnh hoặc BE qua proxy/upstream), và ngược lại khi trả response.
- **Kiểm tra file cấu hình:** Luôn chạy `nginx -t` để kiểm tra cú pháp trước khi `reload/restart` service, tránh làm sập server đang chạy vì lỗi cấu hình.
- **Log:** Theo dõi log của NGINX (`/var/log/nginx/access.log`, `/var/log/nginx/error.log`) để debug khi request không đến được backend như mong đợi.
- **Self-signed SSL chỉ để test local:** Không bao giờ dùng chứng chỉ tự ký cho môi trường production; trên server thật luôn dùng chứng chỉ từ một CA được tin cậy (ví dụ Let's Encrypt qua Certbot).
- **Đây là bước ghép nối các buổi trước:** NGINX chính là thành phần đứng trước toàn bộ hệ thống Docker Compose (Buổi 6, 7) của nhóm để lộ ra ngoài internet một cách an toàn, có khả năng chịu tải — chuẩn bị sẵn sàng cho việc triển khai lên server thật.

Chúc các bạn học tập hiệu quả và thành công!